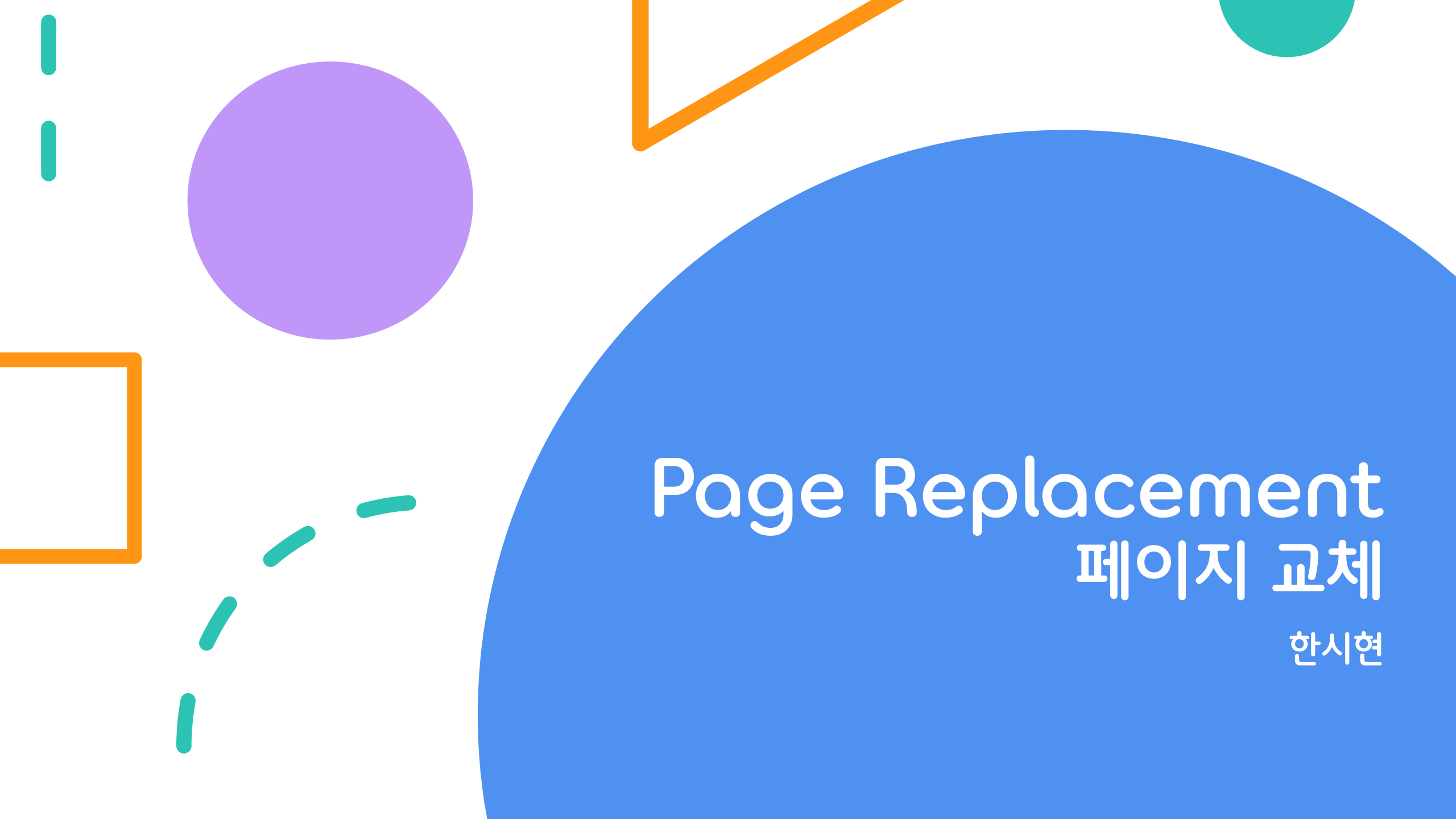


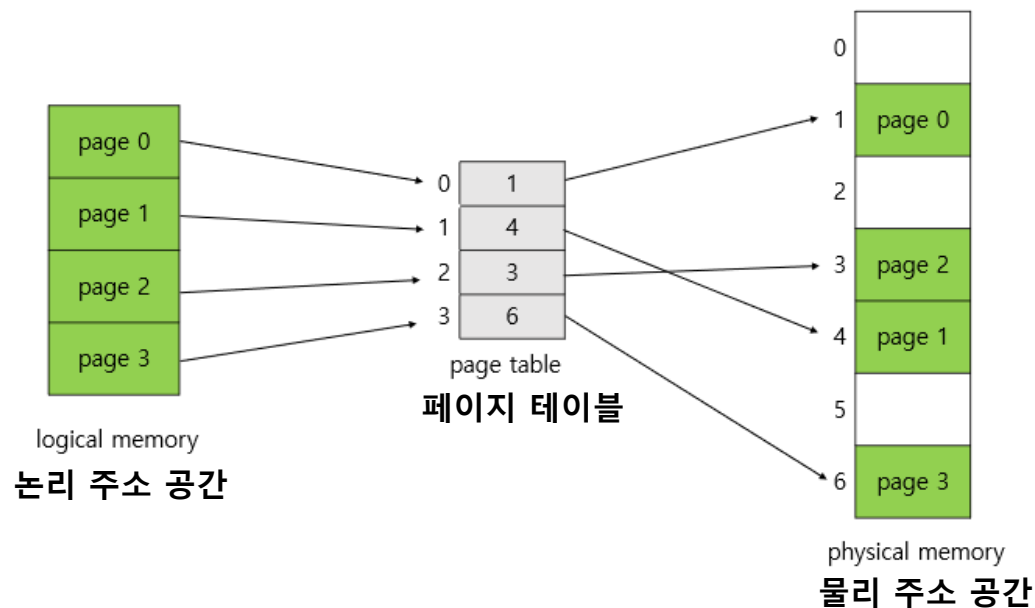


Page Replacement

페이지 교체

한시현

간단 복습 : 페이징 (Paging)

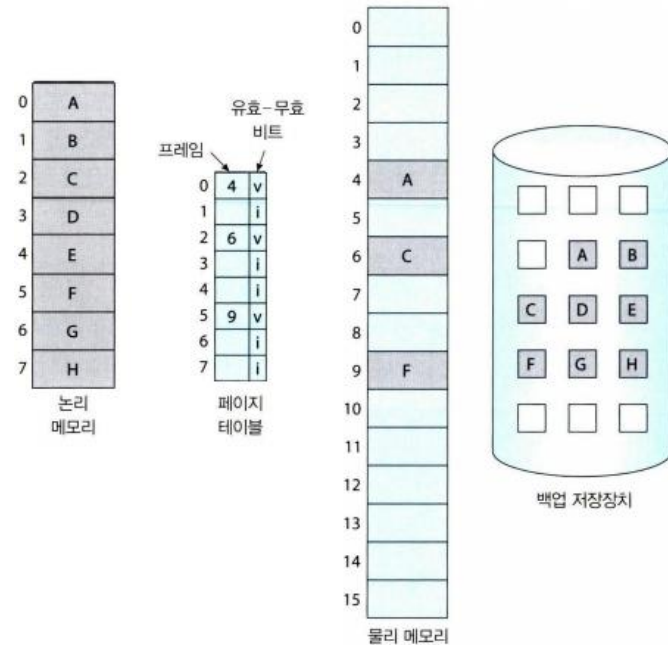


- 물리 주소 공간을 프레임 단위로 자르고, 프로세스의 논리 주소 공간을 페이지 단위로 자른 뒤 각 페이지를 프레임에 할당하는 가상 메모리 관리 기법
- 페이지 테이블을 통해 논리 주소의 페이지 번호를 물리 메모리의 프레임 번호와 매핑
- 메모리 사이에 할당하기 힘든 작은 공간이 생기는 외부 단편화 방지 가능
- 이러한 가상 메모리는 프로그램의 일부만 메모리에 적재하여 실제 물리 메모리 크기보다 더 큰 프로세스를 실행할 수 있게 하는 기술이다.

요구 페이징 (Demand paging)

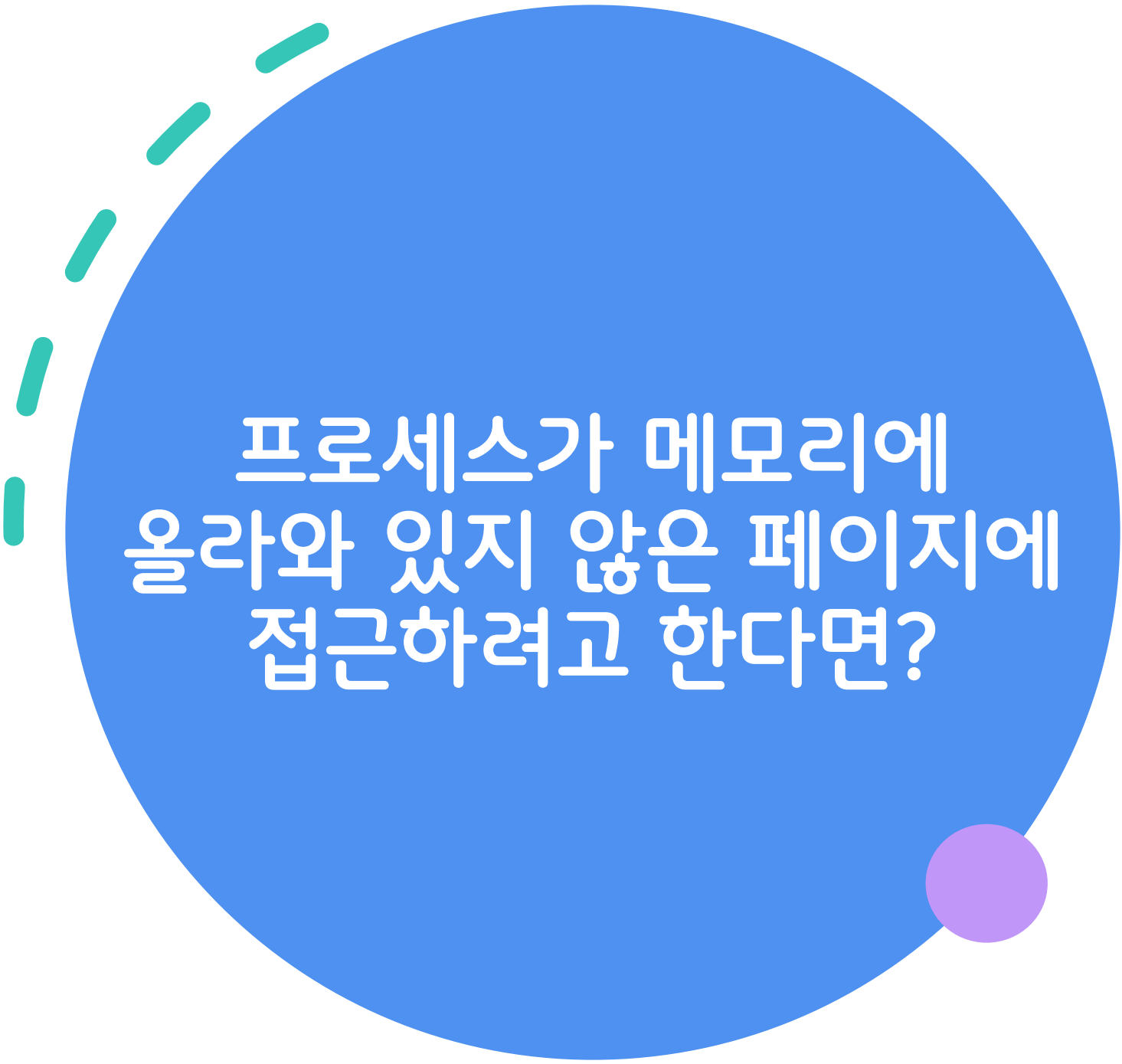
- 프로세스를 메모리에 적재할 때 처음부터 모든 메모리를 적재하지 않고 필요한 페이지만을 메모리에 적재하는 기법 (이름 그대로 실행에 요구되는 페이지만 적재하는 기법이다.)
 - 필요한 프로그램의 일부만 적재하기 때문에 효율적이다.
- 프로세스가 실행되는 동안 일부 페이지는 메모리에 있고, 일부는 보조 저장 장치에 있다.
 - 따라서, 이 둘을 구별하기 위해서 **유효-무효 비트 기법**이 사용될 수 있다.

유효-무효(valid-invalid) 비트 기법



- 물리 메모리에 A, C, F가 있는데 백업 저장장치 (보조 저장장치)에도 A, C, F가 있는 이유?
- 앞서 말했듯, 프로세스가 실행되는 동안 일부 페이지는 메모리에 있고, 일부는 보조 저장 장치에 있기 때문!

- 페이지 번호가 물리 메모리의 특정 프레임과 매핑되어 있음 -> 유효 비트 (or 유효 비트 1)
- 페이지가 물리 메모리와 매핑되지 않음 -> 무효 비트 (or 유효 비트 0)
 1. 페이지 번호가 프로세스의 논리 주소 공간에는 존재하나, 물리 메모리에 올라오지 않은 경우 (유효한 페이지에 대한 참조. But, 보조 저장 장치에 존재하는 경우)
 2. 페이지 번호 자체가 유효한 논리 주소 공간 범위 밖인 경우 (무효한 페이지에 대한 참조)



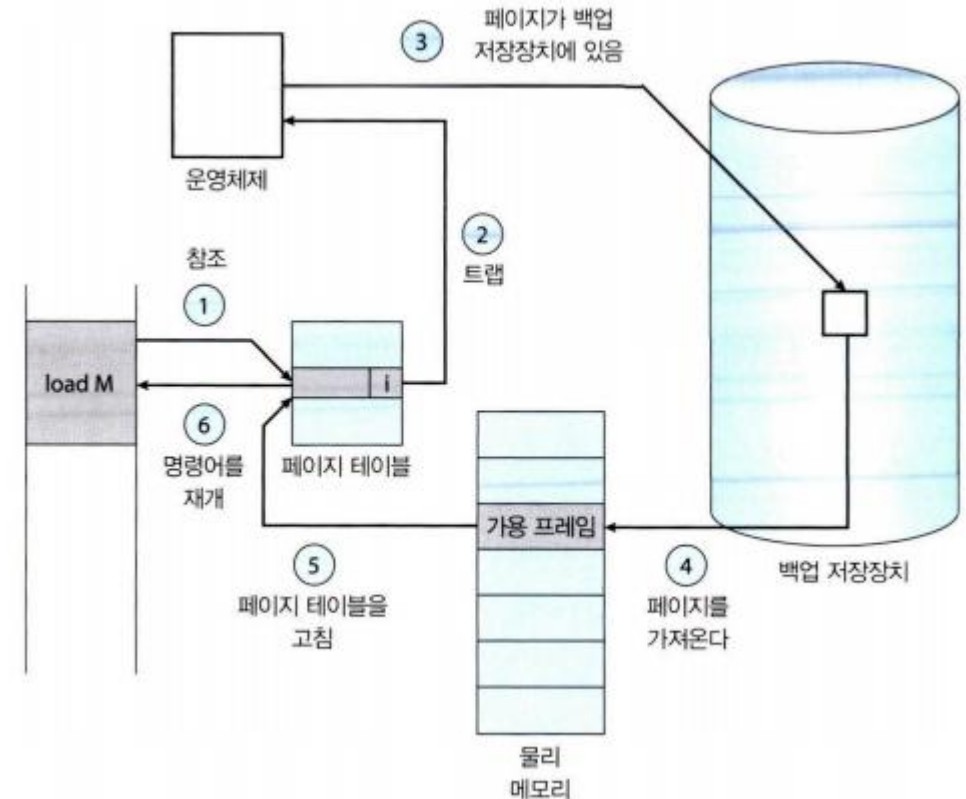
프로세스가 메모리에
올라와 있지 않은 페이지에
접근하려고 한다면?

페이지 폴트 트랩 발생!

- 페이징 하드웨어(MMU, 페이지 테이블, TLB)가 페이지 테이블을 이용한 주소 변환 과정(논리 주소 → 물리 주소)에서 무효 비트를 발견하고 운영체제에 트랩을 건다.
- 페이지 폴트 (Page Fault) : CPU가 어떤 논리 주소의 페이지를 참조했는데, 그 페이지가 현재 물리 메모리(RAM)에 없는 경우 발생하는 사건
- 페이지 폴트 트랩 (Page Fault Trap) : 그 사건(페이지 폴트)을 OS에 알리기 위해 발생하는 인터럽트
 - 페이징 하드웨어(MMU)는 CPU가 낸 논리 주소를 물리 주소로 변환하는 역할까지만 할 수 있다.
 - 그런데 변환 과정에서 무효 비트를 만나면, 이 페이지는 RAM에 없다는 의미이므로 주 기억장치까지만 직접 접근이 가능한 하드웨어 혼자서는 뭘 더 할 수가 없다. (보조 기억 장치에 직접 접근이 불가하므로)
 - 그래서 하드웨어가 CPU 대신 OS에 도와달라는 신호를 보낸다. (OS는 I/O 시스템을 통해 접근할 수 있다.)
 - 트랩이 걸리면 CPU가 수행하던 명령은 일시 중지되고, 제어가 OS에게로 넘어간다.

페이지 폴트를 처리하는 과정

- 1. 프로세스에 대한 내부 테이블을 검사해서 해당 메모리 참조가 유효-무효인지를 확인한다.
- 2. 무효한 페이지에 대한 참조라면 해당 프로세스는 중단한다. 유효한 참조인데 아직 페이지가 메모리에 올라오지 않은 경우라면, 보조 저장 장치(그림에서는 백업 저장장치)로부터 그것을 가져와야 한다.
- 3. 빈 공간(가용 프레임)을 찾는다.
- 4. 보조 저장 장치에 새롭게 할당된 프레임으로 해당 페이지를 읽어 들이도록 요청한다.
- 5. 보조 저장 장치 읽기가 끝나면, 해당 페이지가 이제는 메모리에 있다는 것을 알리기 위해 테이블을 갱신하고, 프로세스가 유지하고 있는 내부 테이블을 수정한다.
- 6. 트랩에 의해 중단되었던 명령어를 다시 수행한다.



순수 요구 페이징 (Pure demand paging)

- 극단적인 경우, 메모리에 페이지가 하나도 올라와 있지 않은 상태에서 프로세스를 실행시킬 수 있다!!
- 이 경우, 프로세스의 첫 명령어를 실행하는 순간부터 페이지 폴트가 발생하게 된다.
- 프로세스의 실행에 필요한 모든 페이지가 메모리에 올라올 때까지 필요할 때마다 페이지 폴트가 발생한다.
- 프로세스의 실행에 필요한 페이지가 어느정도 적재되고 나면 페이지 폴트 발생 빈도가 떨어진다.
- 이렇게 어떤 페이지가 필요해지기 전까지 결코 그 페이지를 메모리로 적재하지 않는 방법이 **순수 요구 페이징**이다. (이런 것도 있구나~)

아무튼 다시 요구 페이징 이야기로 돌아와서...

- 앞서 말한 요구 페이징 시스템이 안정적으로 동작하려면 두 가지 중요한 문제를 해결해야 한다고 한다..
- 바로 페이지 교체와 프레임 할당이다.
- 우리는 이 중 **페이지 교체**에 대해서 알아볼 것이다!!

페이지 교체가 필요한 이유

- 가상 메모리를 통해 실제 물리 메모리보다 큰 프로세스를 실행할 수 있다지만, 그래도 물리 메모리의 크기는 한정되어 있다.
- 요구 페이징 기법으로 페이지들을 적재하다 보면 언젠가 메모리는 가득 차게 될 것이다..
- 따라서 OS는 프로세스들이 한정된 메모리를 효율적으로 이용할 수 있도록 기존 메모리(RAM)에 적재된 불필요한 페이지를 선별하여 보조 저장 장치(HDD, SSD)로 내보내야 하고, 필요한 페이지를 불러와 프레임에 매핑해야 한다.
- 이 과정에서 어떤 페이지를 교체할지 결정하는 것이 **페이지 교체 알고리즘**이다.



페이지 교체 알고리즘

페이지 교체 알고리즘

- 보조 저장장치 I/O가 매우 비용이 많이 든다는 것을 고려할 때, (RAM 접근 속도 대비 디스크 접근 속도는 수천~수만 배 느리다고 함) 좋은 페이지 교체 알고리즘을 통한 요구 페이지 효율성 개선은 시스템의 전체 성능에 크게 기여할 수 있다.
- 그렇다면 좋은 페이지 교체 알고리즘이란 무엇인가?
 - 일반적으로 **페이지 폴트를 적게 일으키는 알고리즘(페이지 폴트율 최소)**을 좋은 알고리즘이라고 평가한다.
 - 페이지 폴트가 발생하면 보조 저장장치로부터 필요한 페이지를 가져와야 하기 때문에 메모리(RAM)에 적재된 페이지에 접근하는 것보다 훨씬 느려지기 때문이다.

특정 페이지 교체 알고리즘을 선택했더니 페이지 폴트가 더 자주 발생했다.
== 보조 저장장치로 내쫓을 페이지를 잘못 골랐다.

페이지 폴트 횟수와 페이지 참조열

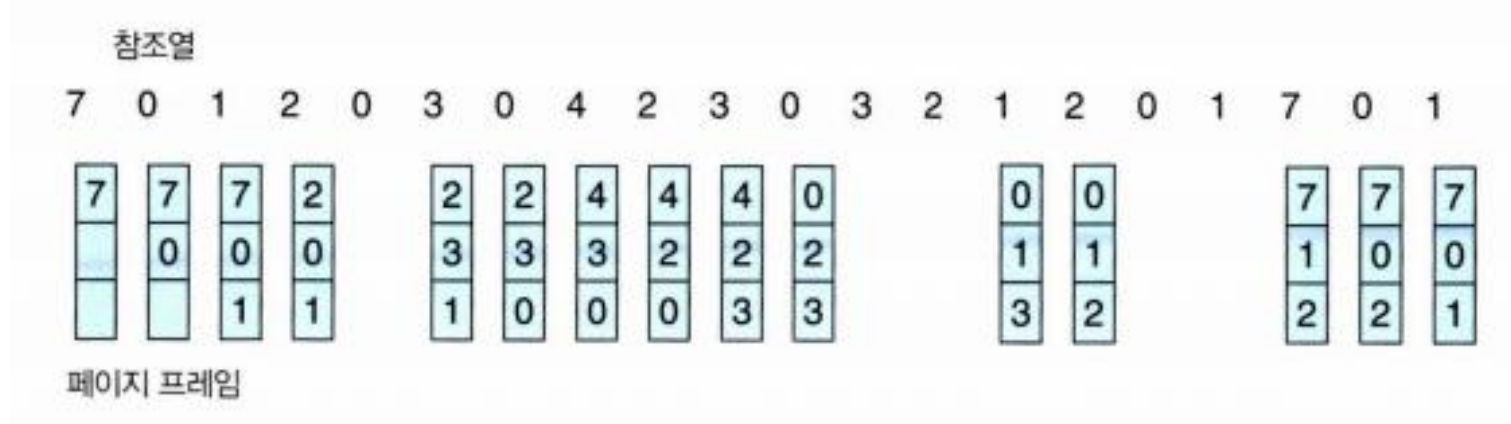
- 페이지 교체 알고리즘의 성능은 **페이지 참조열**에 대해 알고리즘을 적용하여 페이지 폴트 발생 횟수를 계산하여 평가한다.
- 페이지 참조열 (page reference string) : CPU가 참조하는 페이지들 중 연속된 페이지를 생략한 페이지열
- 예를 들어, [1, 1, 1, 2, 2, 3, 3, 3, 5, 3, 6, 6, 7] 순서로 페이지에 접근했다면
- 연속된 페이지를 생략한 페이지열 [1, 2, 3, 5, 3, 6, 7] 이 **페이지 참조열**이다.
- 연속된 페이지를 생략하는 이유는?
- 중복된 페이지를 참조하는 것은 어차피 페이지 폴트를 발생시키지 않기 때문 (어차피 메모리에 올라와있을 테니)
- 페이지 교체 알고리즘 평가 시 우리가 고려할 것은 오직 페이지 폴트의 발생 횟수이다!!



대표적인 페이지 교체 알고리즘

FIFO (First In First Out) 페이지 교체

- 이름 그대로 메모리에 가장 먼저 올라온 페이지부터 내쫓는 방식 (가장 오래 머물렀으면 나가라)
- 페이지가 올라온 시간을 기록하거나, 페이지들이 올라온 순서대로 큐에 넣어서 가지고 있어도 된다.
 - 다만, 시간을 기록하는 방법은 LRU의 계수기(counter) 방식과 유사해져 FIFO의 장점인 단순성이 사라진다.
- 3개의 프레임을 가진 메모리와 페이지 참조열이 다음과 같을 경우 [7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1]

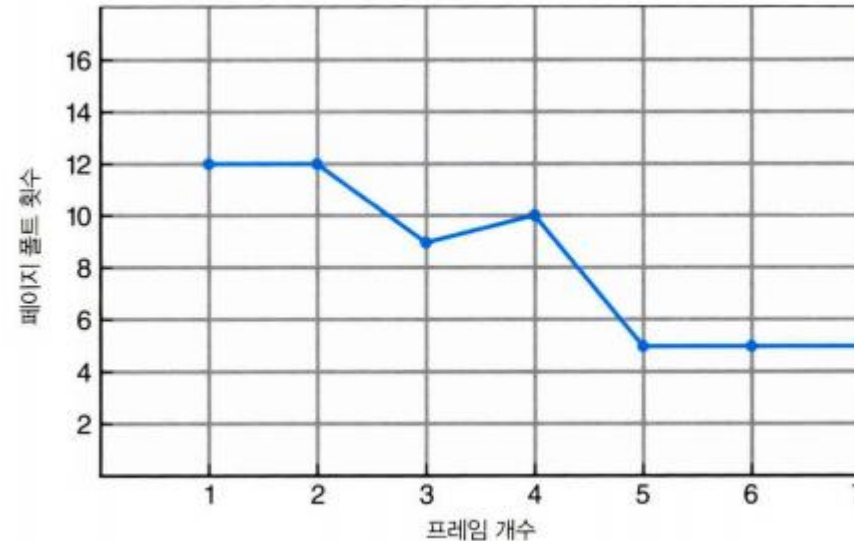


- 초기 적재될 때 발생하는 페이지 폴트를 포함하여 **총 15번의 페이지 폴트 발생**
- 0, 1과 같이 활발하게 사용되는 페이지가 많으면 페이지 폴트율이 높아지고 실행 속도가 떨어진다.
- 다만 이것이 잘못된 실행을 야기하지는 않는다. FIFO 알고리즘에서 발생할 수 있는 심각한 문제점은...

FIFO 문제점 - Belady의 모순

Page Requests	3	2	1	0	3	2	4	3	2	1	0	4
Newest Page	3	2	1	0	3	2	4	4	4	1	0	0
		3	2	1	0	3	2	2	2	4	1	1
Oldest Page			3	2	1	0	3	3	3	2	4	4

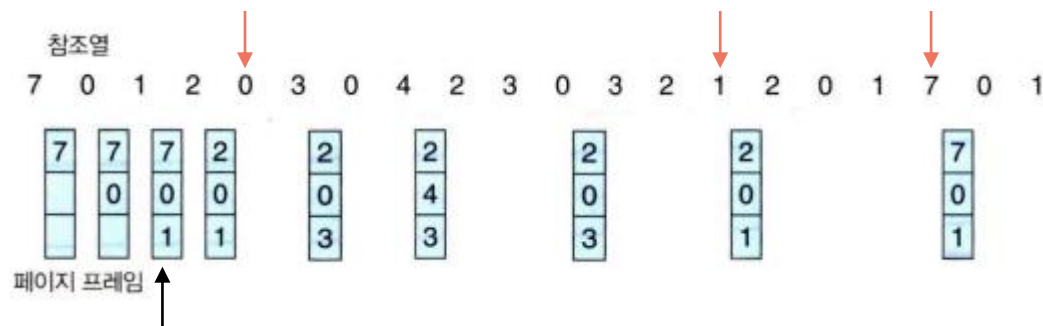
Page Requests	3	2	1	0	3	2	4	3	2	1	0	4
Newest Page	3	2	1	0	0	0	4	3	2	1	0	4
		3	2	1	1	1	0	4	3	2	1	0
			3	2	2	2	1	0	4	3	2	1
Oldest Page				3	3	3	2	1	0	4	3	2



- 참조열이 **[3, 2, 1, 0, 3, 2, 4, 3, 2, 1, 0, 4]** 인 경우를 가정하겠다.
- 위 사진을 보면 3개의 프레임을 할당했을 때 페이지 폴트가 9번 일어났는데, 4개의 프레임을 할당했을 때 페이지 폴트가 10번 일어나는 것을 확인할 수 있다.
- Belady의 모순은 이처럼 프로세스에 프레임을 더 주었는데 페이지 폴트율을 오히려 증가하는 현상을 말한다.

최적 (Optimal) 페이지 교체 (OPT)

- Belady의 모순이 가져온 결과 중 하나는 바로 최적 교체 정책에 대한 탐색이었다.
- 최적 교체 정책이란? 모든 알고리즘보다 낮은 페이지 폴트율을 가지고, Belady의 모순이 발생하지 않는 것.
- 그러한 정책은 존재했고, 이것은 **앞으로 가장 오랫동안 사용되지 않을 페이지를 찾아 교체하는 방법**이었다.
- 3개의 프레임을 가진 메모리와 페이지 참조열이 다음과 같을 경우 [7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1]



이 시점에서 7, 0, 1 중 가장 오랫동안 사용되지 않을 페이지는 7이다. 7을 교체한다.

- 보기엔 훌륭하나, 이 알고리즘은 실제 구현은 어렵다.
- 프로세스가 앞으로 메모리를 어떻게 참조할 것인지를 미리 알아야 하기 때문.
- 이 최적 알고리즘은 실제 사용되지는 않고, 비교 연구 목적을 위해서 사용된다.

OPT 문제점 - 사실상 구현이 불가능

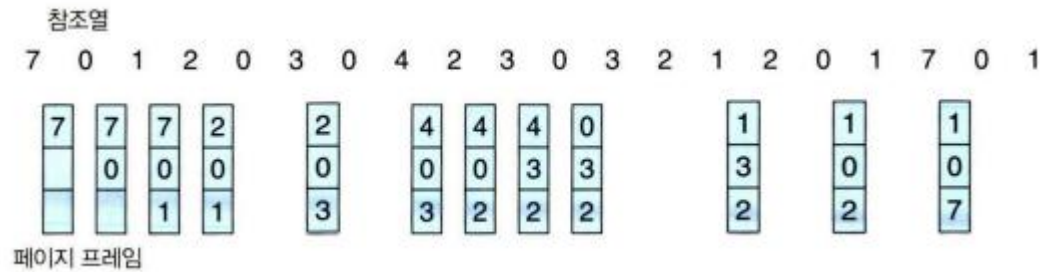
- 보기엔 훌륭하나, 이 알고리즘은 실제 구현은 어렵다.
- 프로세스가 **앞으로 메모리를 어떻게 참조할 것인지를** 알아야 하기 때문. (미래 예측..?)
- 그렇기 때문에 이 최적 알고리즘은 실제 사용되지는 않고, 비교 연구 목적을 위해서 사용된다.
 - 이 새로운 정책은 최적 알고리즘과 비교했을 때 평균적으로 몇 % 차이가 난다 등

LRU(Least Recently Used) 페이지 교체

- 최적 알고리즘이 불가능하다면, 최적 알고리즘의 근사 알고리즘을 가능할 것이다.
- 최근의 과거를 가까운 미래의 근사치로 본다면, **가장 오랜 기간동안 사용되지 않은 페이지를 교체**할 수 있다.
- 가장 오랫동안 사용되지 않을 페이지를 교체하는 알고리즘 구현이 어렵다면, 가장 오랫동안 사용되지 않은 페이지를 교체하는 알고리즘은 구현이 가능하다는 것이다.
- 이 기법이 LRU (Least Recently Used) 알고리즘이다.
- **최근에 사용되지 않은 페이지는 앞으로도 사용되지 않을 것이라는 가정** 하에 만들어진 알고리즘이다.

LRU(Least Recently Used) 페이지 교체

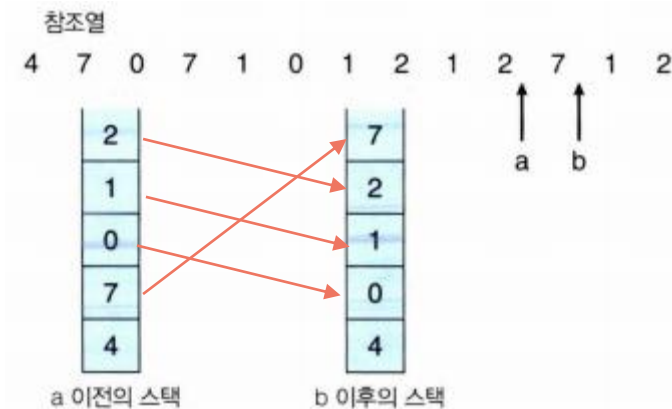
- 3개의 프레임을 가진 메모리와 페이지 참조열이 다음과 같을 경우 [7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1]



- 초기 적재될 때 발생하는 페이지 폴트를 포함하여 **총 12번의 페이지 폴트 발생**
- 앞서 동일한 페이지 참조열에서 15번의 페이지 폴트를 발생시킨 FIFO 알고리즘보다 우수하다.
- 이렇듯, LRU 페이지 교체 알고리즘은 좋은 알고리즘으로 인정받고 있다.
- 프레임들을 최근 사용된 시간 순서로 파악할 수 있어야 하기 때문에 2가지의 구현 방법이 존재한다.

LRU 구현 방법

- 1. 계수기 (counter) 방식
 - 각 페이지 항목마다 사용 시간 필드를 넣고, cpu에 논리적인 시계나 계수기를 둔다.
 - 각 페이지 테이블을 조회할 때마다 마지막으로 참조된 시간을 기록한다.
 - 페이지 교체 시, 가장 오래된 시간 값을 가진 페이지를 선택한다.
- 2. 스택 (stack) 방식
 - 페이지 번호의 스택을 유지하는 방법.
 - 페이지가 참조 될 때마다 페이지 번호는 스택 중간에서 제거되어 스택 꼭대기에 놓인다.
 - 이렇게 되면 꼭대기는 항상 최근에 사용된 페이지가 위치하고, 밑바닥에는 가장 오랫동안 사용되지 않은 페이지가 위치한다.
 - 페이지 교체 시, 스택 맨 아래 페이지를 선택한다.



LRU 문제점 - 구현이 어려움

- 순수 LRU를 구현할 경우, 모든 페이지에 대해 마지막으로 참조된 시간을 기록하거나(계수기 방식) 스택을 유지해서 참조될 때마다 위치를 갱신해야 한다(스택 방식).
- 다만, 이러한 방식은 메모리 접근이 일어날 때마다 이 정보들(계수기 값, 스택)을 갱신해야 해서 막대한 하드웨어 오버헤드를 유발하고, 별도의 하드웨어(계수기)나 추가적인 메모리 공간(스택)을 요구한다.
- 그렇다면 LRU에 가까운 선택을 하고 유사한 행동을 보이는 정책을 구현할 순 없을까?

LRU 근사(Approximation) 페이지 교체

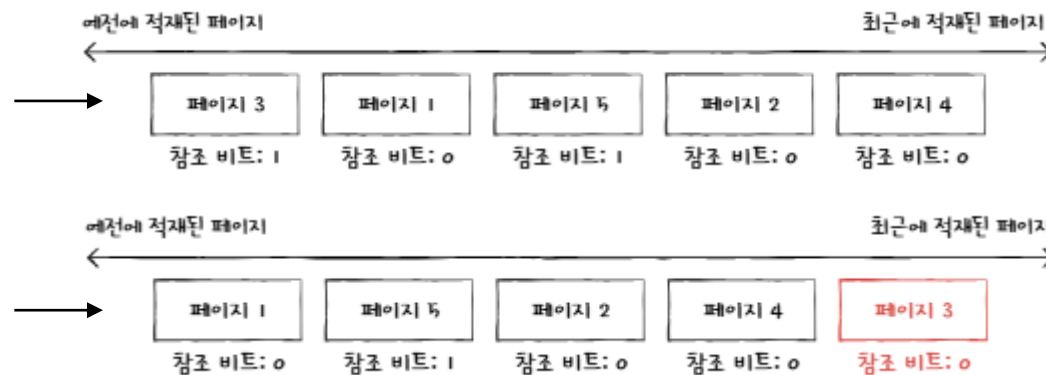
- 실제 OS에서는 순수 LRU가 아닌, 이를 비슷하게 흉내낼 수 있는(근사) 알고리즘을 사용한다.
 - 여기에는 **참조 비트(reference bit)**라고 하는 약간의 하드웨어 지원이 동반된다.
 - 시스템의 각 페이지마다 하나의 참조 비트를 가지며, 이 참조 비트는 메모리 어딘가에 존재한다.
 - 구현에 따라 각 프로세스의 페이지 테이블에 있을수도 있고, 또는 어딘가에 배열 형태로 존재할 수도 있다.
 - 처음 모든 참조 비트는 OS에 의해 0으로 채워지고, 프로세스가 실행되면서 참조되는 페이지의 비트는 하드웨어가 1로 세팅한다.
 - 이후 얼마가 지나면 페이지의 어떤 페이지가 사용되었고, 어떤 페이지가 한 번도 사용되지 않았는지를 알 수 있다.
 - 이 정보가 LRU 근사 알고리즘의 기본이 된다.
-
- LRU 근사 알고리즘의 대표적인 예들은 다음과 같다.
 - 2차 기회 알고리즘
 - Clock 알고리즘
 - 개선된 2차 기회 알고리즘

LRU 근사 - 2차 기회 알고리즘

- 2차 기회 알고리즘의 기본은 FIFO 교체 알고리즘이다.
 - 자주 참조되는 페이지가 먼저 적재되었다는 이유로 내쫓길 수 있는 문제가 있는 FIFO 알고리즘의 부작용을 어느정도 개선했다.
- 기본적으로는 FIFO 처럼 메모리에 가장 오래 머무른 페이지를 대상으로 내보낼 페이지를 선별한다.
- 차이가 있다면, 페이지가 선택될 때 마다 참조 비트를 확인하여 참조 비트가 0이면 페이지를 교체하고, 1이면 다시 한번 기회를 주고 참조 비트를 0으로 만든 뒤 현재 시간을 적재 시간으로 설정한다.
- 참조 비트가 1이라는 의미는 cpu가 접근한 적이 있다는 것이므로, 메모리에 가장 오래 머물렀다고 해도 한 번의 기회를 주는 것.
- 참조 비트가 0이라는 의미는 가장 오래된 페이지이면서 사용되지 않은 페이지라고 할 수 있기 때문에 보조 저장장치로 추방.

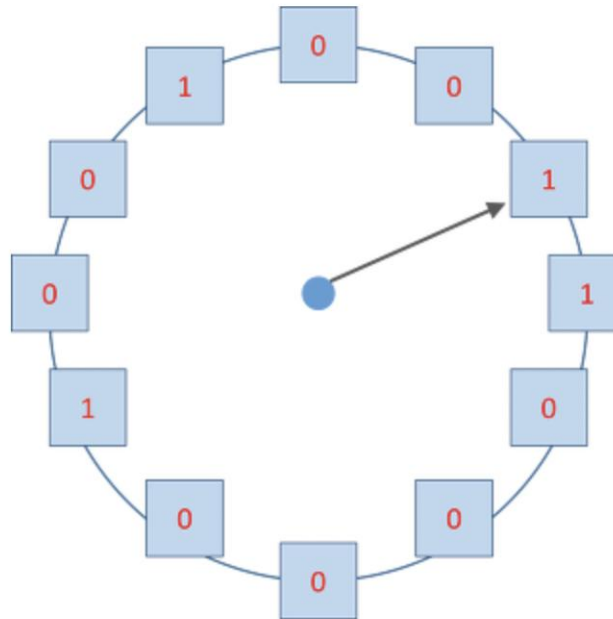
현재 페이지가 3,1,5,2,4 순으로 적재.
원래라면 3을 내보내야 하지만 참조
비트가 1이므로 이를 0으로 변경 후
최근에 적재된 페이지로 간주.

다음 페이지 1의 경우는 참조 비트가
0이므로, 보조 저장장치로 내보내고
1이 적재되었던 프레임에 새 페이지
적재



LRU 근사 - Clock 알고리즘

- 2차 기회 알고리즘을 원형 큐로 효율적으로 구현한 방식.
- 프레임들을 원형으로 배열하고, 포인터(시계 바늘)가 돌면서 참조 비트 확인
- 원리는 같으나, 실제 OS에서는 Clock이 더 널리 사용된다.



LRU 근사 - 개선된 2차 기회 알고리즘

- 참조 비트와 변경 비트 2가지를 사용해서 2차 기회 알고리즘을 좀 더 개선한 알고리즘
- 2개의 비트를 조합하여 다음 4가지 등급이 가능하다.
- (참조 비트 R, 변경 비트 M)
- (0, 0) : 최근에 사용되지도, 변경되지도 않은 경우 -> 1순위, 교체하기 최고
- (0, 1) : 최근에 사용되지는 않았지만 변경은 된 경우 -> 2순위 (최근 사용되지는 않았지만, 페이지가 변경된 상태이기 때문에 RAM에서 내보내기 전에 변경된 내용을 보조 저장장치에 기록해야 한다 -> I/O 비용 발생)
- (1, 0) : 최근에 사용은 되었으나 변경은 되지 않은 경우 -> 3순위 (최근 참조되었으니 다시 쓸 가능성 있음)
- (1, 1) : 최근에 사용도 되었고 변경도 된 경우 -> 4순위, 교체하기 최악
- 개선된 2차 기회 알고리즘은 2차 기회와 달리 I/O 비용까지 고려할 수 있다! (성능이 더 좋다!)

계수-기반 페이지 교체 알고리즘

- LFU (least frequently used) 알고리즘
 - 지금까지의 참조 횟수를 기반으로, 가장 적게 사용된 페이지를 교체
 - 적게 참조된 페이지는 앞으로도 덜 쓰일 것이라는 가정 하에 만들어진 알고리즘
 - 초기에 특정 페이지를 많이 사용하고, 이후에 이 페이지를 사용하지 않는 경우 판단이 빗나갈 수 있다.
- MFU (most frequently used) 알고리즘
 - 지금까지의 참조 횟수를 기반으로, 가장 많이 사용된 페이지를 교체
 - 참조 횟수가 작은 페이지일수록 가장 최근에 올라온 것이고, 앞으로 계속 쓰일 가능성이 높다는 가정 하에 만들어진 알고리즘
 - 반대로, 많이 참조된 페이지는 이제 필요 없을 수 있다.
- 위 두 방법은 잘 쓰이지는 않는데, 알고리즘 구축 시에 비용(참조 횟수 기록/갱신)이 많이 들고, 최적 페이지 교체 정책을 제대로 근사하지 못하기 때문

페이지 교체 알고리즘 총정리

- 기본 알고리즘
 - FIFO: 가장 먼저 들어온 페이지부터 교체
 - OPT: 앞으로 가장 오래 쓰이지 않을 페이지 교체 (이론적 최적, 실제 구현 불가)
 - LRU: 가장 오래 참조되지 않은 페이지 교체
- LRU 근사 알고리즘
 - 2차 기회 (Second-Chance): FIFO + 참조 비트
 - Clock: 2차 기회의 효율적 구현 (원형 큐 + 포인터)
 - 개선된 2차 기회 (ESC): Clock + 참조/변경 비트
- 사용 빈도 기반 알고리즘
 - LFU: 참조 횟수가 가장 적은 페이지 교체
 - MFU: 참조 횟수가 가장 많은 페이지 교체
- 그 외
 - NRU, Working Set, WSClock, 2-list (현대 Linux가 채택한 알고리즘) 등
 - 실제 운영체제에서는 이런 LRU 변형 알고리즘들이 쓰임



감사합니다